

PROJET DE RECHERCHE — CMI INFORMATIQUE

Création d'un Assistant Intelligent pour les Cours de Première Année

Public cible : Tous

Encadrant : Yannick Estève

A. Louvet L. Ouelhadj T. De Faveri-Serre C. Cloupet H. Baudey

Années universitaires 2025–2027

Ce projet vise d'une part à produire des contributions de recherche sur les grands modèles de langage (LLM), la génération augmentée par récupération (RAG), la recherche vectorielle et les frameworks d'orchestration dans le contexte des assistants pédagogiques, en les rendant accessibles quelle que soit la familiarité préalable avec ces concepts ; et d'autre part à construire un assistant conversationnel ancré dans le RAG, capable de répondre aux questions des étudiants à partir de leurs propres documents de cours.

Table des matières

1	État de l'Art : Définitions et Contexte	3
1.1	Introduction à l'Intelligence Artificielle	3
1.1.1	Bref historique	3
1.1.2	Qu'est-ce que l'Intelligence Artificielle ?	3
1.2	Les Grands Modèles de Langage (LLM)	4
1.2.1	L'architecture Transformer	4
1.2.2	Un outil polyvalent	4
1.2.3	Limites : Hallucinations et Frontières de la Connaissance	5
1.2.4	Biais et Dégradation par Boucle de Rétroaction via le RLHF	5
1.3	Récapitulatif	6
1.4	Génération Augmentée par Récupération (RAG)	6
1.4.1	Principe et Motivation	6
1.4.2	Architecture du RAG	7
1.4.3	Variantes et Évolution du RAG	7
1.5	Recherche Vectorielle et Indexation	7
1.5.1	Qu'est-ce qu'un Embedding ?	7
1.5.2	Comment les Embeddings sont-ils Appris ?	8
1.5.3	Mesurer la Similarité entre Vecteurs	8
1.5.4	Indexation et Recherche Approchée du Plus Proche Voisin	8
1.5.5	Synthèse	9
2	Présentation du Projet	10
2.1	Objectifs Pédagogiques	10
2.2	Les Assistants IA en Contexte Éducatif	10
2.2.1	Panorama des Travaux Existants	10
2.2.2	Efficacité et Défis	10
2.3	Synthèse et Positionnement de Notre Projet	12

1. État de l'Art : Définitions et Contexte

1.1. Introduction à l'Intelligence Artificielle

1.1.1 Bref historique

L'idée de machines capables de penser n'est pas nouvelle. Dès 1950, Alan Turing proposa son célèbre *Test de Turing* comme critère d'intelligence machine [17] : si une machine peut soutenir une conversation indiscernable de celle d'un être humain, on peut, dans un certain sens, dire qu'elle pense. Le terme « Intelligence Artificielle » lui-même fut inventé en 1956 par John McCarthy lors de la conférence de Dartmouth, considérée comme l'acte fondateur du domaine.

Les décennies suivantes alternèrent entre périodes d'optimisme et « hivers de l'IA » — des intervalles de réduction des financements et de l'intérêt, causés par l'écart entre les promesses ambitieuses et les résultats modestes. Le paysage changea radicalement dans les années 1980 et 1990 avec l'essor de l'*apprentissage automatique* : plutôt que de coder des règles en dur dans un programme, les chercheurs commencèrent à concevoir des systèmes capables d'*apprendre* des patterns directement à partir des données. Ce changement de paradigme posa les fondements de tout ce qui suivit.

Le véritable tournant arriva en 2012, quand un réseau de neurones profond appelé Alex-Net [7] écrasa la concurrence lors du défi de reconnaissance visuelle ImageNet, démontrant que l'*apprentissage profond* — des réseaux de neurones à nombreuses couches entraînés sur de grands jeux de données — pouvait atteindre des performances bien supérieures aux méthodes classiques. À partir de ce moment, l'apprentissage profond devint l'approche dominante dans presque tous les sous-domaines de l'IA.

1.1.2 Qu'est-ce que l'Intelligence Artificielle ?

Le terme *Intelligence Artificielle* est l'une des expressions les plus utilisées — et les plus galvaudées — du paysage technologique actuel. Avant d'aborder les systèmes spécifiques au cœur de ce projet, il convient de prendre du recul pour comprendre ce qu'est réellement l'IA, d'où elle vient et pourquoi elle est devenue si centrale en informatique moderne.

Dans son essence, l'Intelligence Artificielle désigne tout système de calcul conçu pour effectuer des tâches qui nécessiteraient normalement l'intelligence humaine — comme reconnaître des objets dans une image, comprendre une phrase prononcée, ou décider du prochain coup dans une partie d'échecs. Une définition plus précise, proposée par Russell et Norvig dans leur manuel de référence [14], décrit un agent IA comme un système qui *perçoit son environnement à travers des capteurs et agit sur lui par des actionneurs afin de maximiser une mesure de performance*.

Une distinction utile est celle entre l'*IA étroite* (ou IA faible) et l'*IA générale* (ou IA forte). L'IA étroite — qui recouvre quasiment tous les systèmes existants aujourd'hui — est conçue pour exceller dans un type de tâche précis : un filtre anti-spam classe les e-mails, un moteur de recommandation suggère des films, un modèle de langage génère du texte. L'IA générale, en revanche, serait un système capable de raisonner sur des domaines arbitraires au niveau humain ; elle reste largement hypothétique et constitue un sujet de recherche à long terme. Tous les systèmes d'IA évoqués dans ce projet relèvent de l'IA étroite.

1.2. Les Grands Modèles de Langage (LLM)

Les grands modèles de langage — LLM, pour *Large Language Models*, car le nom complet est un peu long — sont bien plus présents dans notre quotidien qu'on ne le pense, ou même qu'on ne l'anticipait il y a encore quelques années. Mais que sont-ils vraiment ? Que recouvrent-ils, et à quoi servent-ils ?

Un grand modèle de langage est, pour faire simple, un modèle mathématique qui génère du texte en prédisant le mot le plus probable dans une suite, un peu comme les « suggestions de mots » qui apparaissent en haut du clavier quand vous tapez un message sur votre téléphone. Mais ne vous y trompez pas : là où votre smartphone peine à prédire autre chose que votre commande de café habituelle, un LLM opère à une échelle franchement vertigineuse. Imaginez que l'on prenne chaque livre, chaque article, chaque ligne de code disponible sur internet, et qu'on les injecte dans un cerveau numérique qui cartographie les relations complexes entre chaque mot et chaque concept. En calculant la probabilité de la suite d'un texte à partir de milliards de paramètres, ces modèles dépassent largement la simple « saisie automatique » pour entrer dans le domaine du raisonnement et de la créativité : ils ne devinent pas des lettres, ils saisissent les nuances de l'intention humaine.

1.2.1 L'architecture Transformer

Au cœur de ce processus se trouve une structure appelée *Transformer* [18]. Imaginez-la comme un système qui permet au modèle d'examiner un paragraphe entier d'un seul coup, plutôt qu'un mot à la fois. Ce mécanisme d'« *attention* » garantit que le modèle se souvient du début d'un texte pendant qu'il en rédige la fin, maintenant un fil conducteur cohérent qui paraît remarquablement humain.

1.2.2 Un outil polyvalent

Parce qu'ils sont des maîtres de la reconnaissance de motifs (*pattern recognition*), leur utilité s'étend bien au-delà de la réponse à des questions triviales ou de la rédaction d'e-mails :

- **Partenaires créatifs** : Ils peuvent générer des slogans marketing ou rédiger de la poésie dans le style d'un romancier du XIX^e siècle.
- **Assistants techniques** : Ils traduisent des langages de programmation complexes aussi facilement qu'ils traduisent le français en anglais.
- **Synthétiseurs de données** : Ils peuvent résumer un document juridique de 50 pages en trois points en un clin d'œil.

En essence, un LLM agit comme un pont entre le monde rigide et binaire des ordinateurs et le monde désordonné et fluide du langage humain. Ils ne « savent » pas les choses comme nous les savons, mais ils sont des imitateurs de premier ordre de la logique, ce qui en fait l'outil le plus polyvalent de l'ère numérique. À mesure que ces modèles continuent d'être perfectionnés, ils passent de simples curiosités à de véritables collaborateurs essentiels, transformant non seulement la façon dont nous écrivons, mais aussi la façon dont nous résolvons des problèmes et interagissons avec la connaissance collective du monde.

1.2.3 Limites : Hallucinations et Frontières de la Connaissance

Malgré leurs performances impressionnantes, les LLM souffrent de plusieurs limitations bien documentées. La plus critique dans un contexte pédagogique est la tendance à produire des *hallucinations* — générer du texte fluide et plausible mais factuellement incorrect ou sans fondement [20, 6].

Les hallucinations surviennent pour plusieurs raisons. Premièrement, les LLM reposent sur des corrélations statistiques dans leurs données d'entraînement plutôt que sur un raisonnement factuel ancré. Deuxièmement, leur connaissance est statique : ils ont une date limite d'entraînement et ne peuvent pas accéder à des informations nouvelles ou spécifiques à un domaine sauf si elles leur sont explicitement fournies. Troisièmement, lorsqu'une requête sort de la zone de connaissance assurée du modèle, celui-ci peut fabriquer une réponse plutôt qu'admettre son incertitude. Selon une enquête de 2024 publiée dans l'*ACM Transactions on Information Systems*, les hallucinations sont catégorisées en deux types principaux : l'*hallucination de facticité* (énoncer des faits incorrects) et l'*hallucination de fidélité* (produire du contenu incohérent avec une source donnée).

Ces limitations sont particulièrement problématiques dans un contexte pédagogique où l'exactitude est essentielle. Un assistant qui fournit en toute confiance des informations erronées sur un concept de cours pourrait activement induire les étudiants en erreur. C'est ce qui motive le recours à l'ancrage documentaire externe via le RAG.

1.2.4 Biais et Dégradation par Boucle de Rétroaction via le RLHF

Une limitation plus subtile mais significative concerne le mécanisme de retour d'information utilisé pour améliorer les LLM. Des techniques comme l'*Apprentissage par Renforcement à partir des Retours Humains* (RLHF) et l'IA Constitutionnelle sont employées pour aligner le comportement du modèle avec les préférences humaines et réduire les sorties nuisibles. Ce système repose sur un principe dans lequel les utilisateurs notent positivement ou négativement les réponses du modèle, qui est ensuite mis à jour en conséquence [11]. Bien que ce processus vise à aligner le comportement du modèle avec les attentes des utilisateurs, il peut produire des effets pervers lorsque ceux-ci soumettent systématiquement des requêtes mal formulées.

Lorsqu'un utilisateur pose une question ambiguë, incomplète ou incohérente, le modèle n'a pas de base fiable sur laquelle produire une réponse pertinente. Il génère alors une réponse statistiquement plausible au regard de l'entrée, mais qui ne correspond inévitablement pas à l'intention non exprimée de l'utilisateur. Celui-ci, percevant la réponse comme hors sujet ou absurde, attribue une note négative — alors que, du point de vue du modèle, la réponse était une complétion raisonnable de la requête donnée. Ce désalignement entre le signal (une note négative) et sa cause réelle (une requête mal formulée plutôt qu'une erreur du modèle) introduit une *supervision corrompue* dans la boucle d'entraînement.

Répétée à grande échelle, cette dynamique pousse le modèle à associer certains patterns de réponse légitimes à une récompense négative et à les désapprendre, entraînant une dégradation mesurable de la qualité générale des réponses au fil du temps. Une étude de Chen et al. [3] a documenté empiriquement ce phénomène pour ChatGPT, observant des changements de comportement significatifs entre les versions du modèle, avec certaines tâches où les versions antérieures surpassaient les versions ultérieures. Les auteurs attribuent en partie cette régression aux effets cumulatifs du bruit dans les retours d'information des utilisateurs dans le pipeline RLHF.

Ce problème est particulièrement pertinent dans un contexte pédagogique, où les étudiants — surtout en première année — peuvent manquer de compétences en formulation de requêtes pour poser des questions précises. Un assistant qui se dégrade sous des retours bruités pourrait ainsi devenir progressivement moins utile pour son public principal, soulignant l'importance de concevoir des mécanismes robustes de collecte de retours et, dans la mesure du possible, de réduire la dépendance aux évaluations non filtrées des utilisateurs pour les mises à jour du modèle.

1.3. Récapitulatif

Un point de départ naturel pour cet aperçu est une question qui semble simple mais s'avère en réalité assez nuancée : qu'est-ce exactement qu'un LLM et en quoi diffère-t-il de la notion plus large d'« IA » ? Le terme *Intelligence Artificielle* (IA) est un concept chapeau englobant toute technique ou système permettant aux machines de reproduire l'intelligence humaine, incluant l'apprentissage automatique, la vision par ordinateur, la robotique et le traitement du langage naturel. Un *Grand Modèle de Langage* (LLM), en revanche, désigne une catégorie spécifique de modèle IA conçu pour comprendre et générer le langage humain. Les LLM sont un sous-ensemble de l'IA, au même titre que le *Machine Learning* (ML) ou le *Deep Learning* (DL), etc. . . ; si tout LLM est un système IA, la réciproque n'est pas vraie.

Concrètement, les LLM sont des réseaux de neurones entraînés sur d'immenses corpus de texte par apprentissage auto-supervisé, fondés sur l'architecture Transformer introduite par Vaswani et al. [18], qui permet aux modèles de capturer les dépendances à longue portée dans le texte grâce aux mécanismes d'auto-attention. L'introduction ultérieure de la série GPT d'OpenAI — GPT-2 en 2019, GPT-3 en 2020, puis GPT-4 — a démontré que l'augmentation de la taille du modèle et des données d'entraînement conduit à des *capacités émergentes* en compréhension et génération du langage [19] : des capacités qui n'apparaissent qualitativement qu'au-delà d'un certain seuil de taille. Les LLM contemporains tels que GPT-4, Claude d'Anthropic, LLaMA de Meta, Gemini de Google et Mistral font preuve d'une remarquable maîtrise dans un large éventail de tâches — de la réponse aux questions et la synthèse à la génération de code et au raisonnement multi-étapes — et se caractérisent par des nombres de paramètres de l'ordre de dizaines ou centaines de milliards.

1.4. Génération Augmentée par Récupération (RAG)

1.4.1 Principe et Motivation

Avant de plonger dans l'architecture technique, il vaut la peine de prendre du recul pour expliquer ce qu'est le RAG et pourquoi il a été développé, afin de rendre cette recherche accessible et intéressante pour tous.

Imaginez demander à un ami très cultivé une question sur un sujet précis — disons, le contenu d'un cours auquel il n'a jamais assisté. Peu importe son niveau d'intelligence, il ne peut tout simplement pas savoir ce qui a été enseigné dans ce cours particulier. Un LLM fait face à exactement la même limitation : il ne connaît que ce qu'il a vu lors de l'entraînement, et n'a aucun moyen de consulter des documents qu'il n'a jamais rencontrés. Imaginez maintenant qu'avant de répondre, votre ami soit autorisé à feuilleter rapidement les notes de cours pertinentes et à utiliser ce qu'il y trouve pour informer sa réponse. C'est en essence ce que fait le RAG : il donne au modèle accès à un ensemble de documents spécifiques au moment de répondre, de sorte que sa réponse est ancrée dans ces sources plutôt que dans sa mémoire générale (et potentiellement obsolète ou incomplète).

Plus formellement, la Génération Augmentée par Récupération (RAG) est un paradigme qui améliore les LLM en les connectant à une base de connaissances externe au moment de l'inférence [8]. Plutôt que de s'appuyer uniquement sur la mémoire paramétrique du modèle, un système RAG récupère d'abord les passages de documents pertinents d'un corpus curé en fonction de la requête de l'utilisateur, puis transmet ces passages comme contexte au LLM, qui génère une réponse ancrée.

Cette approche répond directement aux limitations décrites dans la section précédente : elle fournit au LLM des informations à jour et spécifiques au domaine, et ancre la génération dans des sources explicites et vérifiables — réduisant significativement le risque d'hallucination. Une enquête spécifiquement focalisée sur le RAG pour les applications pédagogiques [1] confirme que cette approche améliore substantiellement la pertinence et l'exactitude factuelle des réponses générées par l'IA dans les contextes académiques.

1.4.2 Architecture du RAG

Un pipeline RAG standard comprend trois étapes :

1. **Indexation (phase développeur)** : Les documents sources (PDF, diaporamas de cours, notes de lecture, etc.) sont découpés en morceaux (*chunks*), encodés sous forme de vecteurs d'embedding à l'aide d'un modèle d'embedding, et stockés dans une base de données vectorielle. (cf. Section 1.4 avec les détails sur la recherche vectorielle et l'indexation).
2. **Récupération (phase utilisateur)** : Lorsqu'un utilisateur soumet une requête, celle-ci est encodée dans le même espace d'embedding. Le système effectue ensuite une recherche par similarité pour récupérer les k morceaux les plus pertinents de la base de données.
3. **Génération** : Les morceaux récupérés sont ajoutés en préfixe à la requête de l'utilisateur comme contexte dans le prompt du LLM. Le LLM génère ensuite une réponse ancrée dans ces informations récupérées.

1.4.3 Variantes et Évolution du RAG

Le domaine a considérablement mûri depuis l'article original sur le RAG. Des enquêtes de 2024–2025 identifient trois paradigmes principaux [4] :

- **RAG Naïf** : Le pipeline de base récupération-puis-génération décrit ci-dessus.
- **RAG Avancé** : Introduit la réécriture de requêtes, le re-classement des résultats récupérés et la récupération itérative pour améliorer la précision.
- **RAG Modulaire** : Une architecture plus flexible où chaque composant (récupérateur, lecteur, mémoire, etc.) peut être échangé ou optimisé indépendamment.

Des travaux plus récents explorent également le **RAG Agentique**, où le modèle peut décider *quand* récupérer et *quelles* sources consulter en fonction de la complexité de la requête, en utilisant une boucle de raisonnement multi-étapes [21].

Pour notre projet, une architecture RAG Naïf ou Avancé est suffisante étant donné le périmètre bien défini et borné du corpus documentaire (supports de cours de première année).

1.5. Recherche Vectorielle et Indexation

1.5.1 Qu'est-ce qu'un Embedding ?

Un *embedding*, dans le contexte de l'intelligence artificielle et des grands modèles de langage (LLM), désigne la transformation d'un élément d'information — un mot, une phrase, un document, ou parfois même une image — en une séquence de nombres réels appelée *vecteur*. Cette représentation numérique permet à la machine de manipuler le langage et les concepts non plus comme du texte symbolique, mais comme des objets mathématiques sur lesquels elle peut effectuer des calculs.

Plutôt que de « comprendre » les mots comme le ferait un humain, le modèle les place dans un *espace vectoriel* de haute dimension (typiquement entre 384 et 4096 dimensions selon le modèle), où chaque élément occupe une position précise. Dans cet espace, la *distance* entre deux vecteurs reflète leur *similarité sémantique* : deux phrases de sens proches auront des vecteurs voisins, même si elles ne partagent aucun mot en commun. C'est le principe qui permet à un système, par exemple, de récupérer le passage « *la photosynthèse permet aux plantes de produire de l'énergie* » à partir de la requête « *comment les plantes génèrent-elles leur énergie ?* ».

1.5.2 Comment les Embeddings sont-ils Appris ?

Ces vecteurs ne sont pas choisis arbitrairement : ils sont *appris* lors de l’entraînement du modèle. Le système lit des quantités immenses de texte et ajuste progressivement ses paramètres de sorte que les vecteurs résultants capturent les régularités du langage. Par exemple, le modèle apprend que « chat » et « chien » apparaissent dans des contextes similaires et finit par leur attribuer des vecteurs voisins, tandis que « voiture » sera plus éloigné.

Des travaux pionniers comme *Word2Vec* [10] et *GloVe* [12] ont montré que ce simple principe suffit à faire émerger des relations sémantiques remarquables. L’exemple canonique d’arithmétique vectorielle, $\vec{\text{roi}} - \vec{\text{homme}} + \vec{\text{femme}} \approx \vec{\text{reine}}$, illustre que l’espace appris encode des concepts (genre, royauté) plutôt que de simples co-occurrences.

Dans les modèles modernes basés sur les Transformers, les embeddings sont également *contextuels* : le vecteur associé à un mot dépend de la phrase dans laquelle il apparaît. Le mot « banque » dans « je me suis assis au bord de la rivière » et dans « j’ai déposé de l’argent à la banque » reçoit ainsi deux représentations distinctes, ce qui résout naturellement les ambiguïtés lexicales. Pour la recherche sémantique, on s’appuie généralement sur des modèles spécialisés comme ceux de la famille **sentence-transformers** [13], qui produisent un *vecteur unique* par phrase ou paragraphe, optimisé pour la comparaison par similarité plutôt que pour la génération de texte.

1.5.3 Mesurer la Similarité entre Vecteurs

Une fois le contenu représenté sous forme de vecteurs, comparer deux textes revient à comparer deux points dans l’espace. Trois métriques sont couramment utilisées :

- **La similarité cosinus**, qui mesure l’angle entre deux vecteurs et est insensible à leur magnitude : de loin la métrique la plus utilisée en recherche sémantique.
- **Le produit scalaire**, plus rapide à calculer mais sensible aux normes des vecteurs ; préféré lorsque les vecteurs ont été normalisés au préalable.
- **La distance euclidienne**, intuitive mais souvent moins discriminante en très haute dimension en raison du « fléau de la dimensionnalité ».

1.5.4 Indexation et Recherche Approchée du Plus Proche Voisin

Lorsqu’un corpus contient des milliers, voire des millions de passages indexés, comparer la requête à chaque vecteur stocké — la recherche dite *exacte* ou « force brute » — devient prohibitivement coûteux. La solution standard consiste à s’appuyer sur des structures d’*indexation* permettant la *recherche approchée du plus proche voisin* (ANN) : on échange une petite perte de précision contre plusieurs ordres de grandeur en rapidité.

Plusieurs algorithmes d’indexation sont devenus des standards :

- **IVF (Index de Fichier Inversé)** : partitionne l’espace en cellules via une étape préalable de regroupement (typiquement *k*-means), puis restreint la recherche aux cellules les plus proches de la requête.
- **HNSW (Monde de Petits Mondes Hiérarchique Navigable)** [9] : construit un graphe multi-couches dans lequel la navigation locale converge rapidement vers le voisinage du vecteur requêté. C’est actuellement l’algorithme de référence dans la plupart des bases de données vectorielles, grâce à son excellent compromis entre rappel et latence.
- **Quantification par Produit (PQ)** : compresse les vecteurs en sous-quantifiant chaque sous-espace, réduisant drastiquement l’utilisation mémoire au prix d’une légère perte de précision.

Ces algorithmes sont implémentés dans des bibliothèques et bases de données dédiées — les *bases de données vectorielles* — les plus répandues dans l’écosystème open-source étant **FAISS**

(Facebook AI Similarity Search), **Chroma**, **Qdrant** et **Weaviate**. Le choix dépend principalement du volume de données, des contraintes de latence et de la nécessité d'un filtrage par métadonnées (par exemple, restreindre la recherche à un cours ou un chapitre spécifique).

1.5.5 Synthèse

En pratique, les embeddings constituent le bloc de construction fondamental de tout système RAG : ils sont utilisés pour interpréter les requêtes, comparer les passages de cours, effectuer la recherche sémantique et alimenter plus largement les mécanismes internes des LLM. On peut les voir comme une « traduction » du langage humain en une géométrie de concepts, où les idées sont organisées dans un espace multidimensionnel que la machine peut explorer et exploiter efficacement. C'est précisément cette géométrie, combinée à un index adapté, qui permettra à notre assistant pédagogique de récupérer en quelques millisecondes les passages de cours pertinents pour répondre à la question d'un étudiant.

2. Présentation du Projet

2.1. Objectifs Pédagogiques

L'objectif est de découvrir les principes fondamentaux de l'intelligence artificielle moderne à travers les grands modèles de langage (LLM) et le système RAG (Génération Augmentée par Récupération). Les étudiants apprendront à interfacer un LLM avec une base documentaire personnalisée pour créer un assistant conversationnel capable de répondre à des questions portant sur leurs cours.

2.2. Les Assistants IA en Contexte Éducatif

2.2.1 Panorama des Travaux Existants

L'application d'assistants IA à l'éducation suscite un intérêt de recherche croissant. Une enquête publiée dans *Applied Sciences* [16] a recensé 47 études sur les chatbots éducatifs basés sur le RAG, la majorité publiées en 2024, témoignant de la croissance rapide de ce domaine.

Plusieurs déploiements réels sont directement pertinents pour notre projet :

- **UniMiB Assistant** (Université de Bicocca, Milan, 2024) [2] : Un assistant étudiant basé sur le RAG, capable de répondre aux questions sur les procédures universitaires, les emplois du temps et les informations administratives. Il indexe les documents officiels de l'université et utilise un LLM pour générer des réponses ancrées dans ces sources.
- **OwlMentor** (Frontiers in Psychology, 2024) [5] : Un environnement d'apprentissage alimenté par l'IA conçu pour aider les étudiants universitaires à comprendre des textes scientifiques. Intégré dans un cours, il offrait un chat basé sur les documents, la génération automatique de questions et la création de quiz.
- **Assistant IA du CS50** (Université Harvard) : Intégré dans un cours d'introduction à l'informatique, un assistant basé sur le RAG s'est avéré capable de fournir des réponses détaillées et contextuellement appropriées, facilitant le tutorat personnalisé à grande échelle.
- **Assistant RAG pour l'enseignement médical** (Nature *npj Digital Medicine*, 2025) [15] : Un assistant pédagogique déployé dans un cours de médecine, où les modèles d'utilisation et les retours des étudiants ont été analysés sur deux cohortes consécutives, démontrant la faisabilité et l'efficacité des assistants basés sur le RAG dans des contextes éducatifs à enjeux élevés.

2.2.2 Efficacité et Défis

Le constat général à travers ces études est que les assistants basés sur le RAG sont efficaces pour fournir des réponses précises et ancrées dans le contexte à des questions spécifiques à un domaine, et que les étudiants les perçoivent positivement comme un complément à l'enseignement humain. Cependant, plusieurs défis sont régulièrement rapportés :

- **Qualité et couverture des documents** : La qualité des réponses de l'assistant est directement bornée par la qualité et la couverture du corpus documentaire indexé. Des documents de cours mal formatés ou incomplets dégradent les performances.
- **Requêtes hors domaine** : Les étudiants posent souvent des questions en dehors du domaine des documents indexés. Les systèmes robustes doivent détecter ces requêtes et refuser gracieusement de spéculer.
- **Difficulté d'évaluation** : Évaluer la correction des réponses générées dans des contextes éducatifs ouverts est non trivial et nécessite souvent une évaluation par des experts humains.

- **Calibration de la confiance des étudiants :** Les étudiants peuvent sur-dépendre des réponses de l'IA ou, à l'inverse, s'en méfier même lorsqu'elles sont correctes. Communiquer l'incertitude et citer les sources est important pour une calibration appropriée de la confiance.

2.3. Synthèse et Positionnement de Notre Projet

Cet état de l'art révèle que la combinaison des LLM et du RAG représente la meilleure pratique actuelle pour construire des assistants conversationnels spécifiques à un domaine. La pile technologique que nous prévoyons d'utiliser — Python, un modèle d'embedding (`sentence-transformers` ou OpenAI), une base de données vectorielle (FAISS ou Chroma) et un framework d'orchestration (LangChain ou LlamaIndex) — est bien validée tant dans la littérature académique que dans les déploiements éducatifs réels.

L'originalité principale de notre projet réside dans sa portée spécifique : construire un assistant adapté aux *supports de cours de première année du CMI Informatique*, qui présentent des caractéristiques particulières (informatique et mathématiques de niveau introductif, contenu de projet en français et en anglais). Les défis que nous anticipons s'alignent avec ceux rapportés dans la littérature : assurer une couverture documentaire suffisante, gérer les questions hors périmètre et calibrer la confiance des étudiants dans les sorties du système.

Références

- [1] Muhammad Afzaal, Jalal Nouri, Aqdas Zia, and Panagiotis Papapetrou. Retrieval-augmented generation (RAG) chatbots for education : A survey of applications. *Computers and Education : Artificial Intelligence*, 7, 2024.
- [2] Luca Maria Aiello, Maria Teresa Baldassarre, Daniela Giordano, and Concetto Spampinato. UniMiB assistant : Designing a student-friendly RAG-based chatbot. *arXiv preprint arXiv :2411.19554*, 2024.
- [3] Lingjiao Chen, Matei Zaharia, and James Zou. How is ChatGPT’s behavior changing over time ? *arXiv preprint arXiv :2307.09009*, 2023.
- [4] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models : A survey. *arXiv preprint arXiv :2312.10997*, 2024.
- [5] Shuai Guo, Yang Song, and Jian Tang. Exploring generative AI in higher education : A RAG system to enhance student engagement with scientific literature. *Frontiers in Psychology*, 15, 2024.
- [6] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. Large language models hallucination : A comprehensive survey. *arXiv preprint arXiv :2510.06265*, 2025.
- [7] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 25, 2012.
- [8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.
- [9] Yu A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4) :824–836, 2020.
- [10] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 26, 2013.
- [11] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems (NeurIPS)*, 35 :27730–27744, 2022.
- [12] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. GloVe : Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, 2014.
- [13] Nils Reimers and Iryna Gurevych. Sentence-BERT : Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.
- [14] Stuart Russell and Peter Norvig. *Artificial Intelligence : A Modern Approach*. Prentice Hall, 3rd edition, 2010.
- [15] Malik Sallam, Nizar Abdullah Salim, Muna Barakat, and Ala’a B. Al-Tammemi. A generative AI teaching assistant for personalized learning in medical education. *npj Digital Medicine*, 8, 2025.

- [16] Shamane Siriwardhana, Rivindu Weerasekera, Elliott Wen, Tharindu Kaluarachchi, Rajib Rana, and Suranga Nanayakkara. Retrieval-augmented generation (RAG) chatbots for education : A survey of applications. *Applied Sciences (MDPI)*, 15, 2025.
- [17] Alan M. Turing. Computing machinery and intelligence. *Mind*, 59(236) :433–460, 1950.
- [18] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [19] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Miculivicius, et al. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022.
- [20] Yue Zhang, Yafu Li, Leyang Cui, Deng Cai, Lemao Liu, Tingchen Fu, Xinting Huang, Enbo Zhao, Yu Zhang, Yulong Chen, et al. Siren’s song in the AI ocean : A survey on hallucination in large language models. *arXiv preprint arXiv :2309.01219*, 2023.
- [21] Penghao Zhao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, Jie Jiang, and Bin Cui. A comprehensive survey of retrieval-augmented generation (RAG) : Evolution, current landscape and future directions. *arXiv preprint arXiv :2410.12837*, 2024.